

ODOO DEVELOPER PROGRAM

Advanced Views & Inheritance

Chapter Four – Odoo 17 Development Tutorials

Ahmed Muhumed

July 26, 2026

- 1 Chapter Four Overview
- 2 Transient Models and Wizard Forms
- 3 Abstract Models
- 4 Python and SQL Constraints
- 5 Extension and Delegation Inheritance
- 6 XPath and Extend Views
- 7 Smart / Stats Buttons
- 8 Advanced Views
- 9 Navigation and Actions
- 10 Summary

Chapter Four Overview

WHAT THIS CHAPTER COVERS

- ▶ Transient models and wizard forms
- ▶ Abstract models
- ▶ Python and SQL constraints
- ▶ Extension and delegation inheritance
- ▶ XPath and view extension
- ▶ Smart / stat buttons
- ▶ Advanced views: Pivot, Kanban, Calendar, Graph, Activity, Gantt, and more

WHY THESE TOPICS MATTER

- ▶ Real Odoo apps are not only form + tree views.
- ▶ Wizards guide users through business processes.
- ▶ Inheritance lets you extend Odoo without rewriting core modules.
- ▶ Advanced views turn records into dashboards, boards, and schedules.
- ▶ Constraints protect data quality at Python and SQL levels.

LEARNING GOALS

- ▶ Build a wizard with a transient model and form view.
- ▶ Create reusable abstract models.
- ▶ Add Python and SQL constraints correctly.
- ▶ Extend models with `_inherit` and delegation (`_inherits`).
- ▶ Extend views with XPath inheritances.
- ▶ Add smart buttons and advanced view types.

Transient Models and Wizard Forms

WHAT IS A TRANSIENT MODEL?

- ▶ A **transient model** stores temporary wizard data.
- ▶ It inherits from `models.TransientModel`.
- ▶ Records are cleaned automatically after some time.

Method Syntax

```
1 class StudentWizard(models.TransientModel):  
2     _name = 'student.wizard'  
3     _description = 'Student Wizard'
```


TRANSIENTMODEL VS MODEL

- ▶ `models.Model`: normal business data (students, invoices, partners).
- ▶ `models.TransientModel`: temporary UI/process data.

```
1 # Permanent
2 class Student(models.Model):
3     _name = 'student.student'
4
5 # Temporary wizard
6 class StudentWizard(models.TransientModel):
7     _name = 'student.wizard'
```

- ▶ Do not store important long-term business data in transient models.

BUILDING A WIZARD MODEL

```
1 class StudentAssignWizard(models.TransientModel):  
2     _name = 'student.assign.wizard'  
3     _description = 'Assign Classroom Wizard'  
4  
5     classroom_id = fields.Many2one('school.classroom', required=True)  
6     student_ids = fields.Many2many('student.student', string='Students')  
7  
8     def action_assign(self):  
9         self.ensure_one()  
10        self.student_ids.write({'classroom_id': self.classroom_id.id})  
11        return {'type': 'ir.actions.act_window_close'}
```

WIZARD FORM VIEW

```
1 <record id="view.student.assign.wizard" model="ir.ui.view">
2   <field name="name">student.assign.wizard.form</field>
3   <field name="model">student.assign.wizard</field>
4   <field name="arch" type="xml">
5     <form string="Assign Classroom">
6       <group>
7         <field name="classroom_id"/>
8         <field name="student_ids" widget="many2many_tags"/>
9       </group>
10      <footer>
11        <button name="action_assign" type="object" string="Assign" class="btn-primary"/>
12        <button string="Cancel" special="cancel"/>
13      </footer>
14    </form>
15  </field>
16 </record>
```

OPENING A WIZARD WITH AN ACTION

```
1 def action_open_assign_wizard(self):
2     return {
3         'type': 'ir.actions.act_window',
4         'name': 'Assign Classroom',
5         'res_model': 'student.assign.wizard',
6         'view_mode': 'form',
7         'target': 'new',
8         'context': {
9             'default_student_ids': [(6, 0, self.ids)],
10        },
11    }
```

- ▶ target='new' opens the wizard as a modal dialog.
- ▶ Context can prefill wizard fields.

WIZARD BEST PRACTICES

- ▶ Keep wizards focused on one job.
- ▶ Validate inputs in the wizard method.
- ▶ Close the wizard with `act_window_close` or return another action.
- ▶ Use `default_*` context keys to prefill values.
- ▶ Remember: transient data is temporary.

Abstract Models

WHAT IS AN ABSTRACT MODEL?

- ▶ An **abstract model** is a reusable mixin.
- ▶ It inherits from `models.AbstractModel`.
- ▶ It does **not** create its own database table.

Method Syntax

```
1 class SchoolMailMixin(models.AbstractModel):  
2     _name = 'school.mail.mixin'  
3     _description = 'School Mail Mixin'
```

WHY USE ABSTRACT MODELS?

- ▶ Share fields and methods across many models.
- ▶ Avoid copy-paste logic.
- ▶ Classic Odoo examples:
 - `mail.thread`
 - `mail.activity.mixin`
 - `portal.mixin`

CREATING AND USING A MIXIN

```
1 class ArchiveMixin(models.AbstractModel):  
2     _name = 'school.archive.mixin'  
3     _description = 'Archive Mixin'  
4  
5     active = fields.Boolean(default=True)  
6  
7     def action_archive(self):  
8         self.write({'active': False})  
9  
10 class Student(models.Model):  
11     _name = 'student.student'  
12     _inherit = ['school.archive.mixin']
```

- ▶ student.student **receives** active **and** action_archive.
- ▶ No table is created for school.archive.mixin.

ABSTRACTMODEL VS MODEL VS TRANSIENTMODEL

- ▶ Model: real persistent business records + table.
- ▶ TransientModel: temporary wizard records + temporary table.
- ▶ AbstractModel: reusable definition only, no own table.

```
1 models.Model  
2 models.TransientModel  
3 models.AbstractModel
```

ABSTRACT MODEL BEST PRACTICES

- ▶ Give mixins clear names ending with `mixin` when useful.
- ▶ Keep them focused (mailing, archiving, rating, etc.).
- ▶ Inherit them with `_inherit = ['my.mixin', ...]`.
- ▶ Do not expect to `search()` on an abstract model itself.

Python and SQL Constraints

WHAT ARE CONSTRAINTS?

- ▶ Constraints protect data integrity.
- ▶ Two main families in Odoo:
 - 1 **Python constraints** (`@api.constrains`)
 - 2 **SQL constraints** (`_sql_constraints`)
- ▶ Use Python for flexible business rules.
- ▶ Use SQL for strong database-level guarantees.

Method Syntax

```
1 from odoo.exceptions import ValidationError
2
3 @api.constrains('age')
4 def _check_age(self):
5     for student in self:
6         if student.age < 5:
7             raise ValidationError("Student age must be at least 5.")
```

- ▶ Runs on create/write when listed fields change.
- ▶ Can read other fields and related values.

MULTI-FIELD PYTHON CONSTRAINT

```
1 @api.constrains('start_date', 'end_date')
2 def _check_period(self):
3     for rec in self:
4         if rec.start_date and rec.end_date and rec.end_date < rec.start_date:
5             raise ValidationError("End date must be after start date.")
```

- ▶ Include every field that should trigger validation.
- ▶ Raise `ValidationError`, not a generic exception.

SQL CONSTRAINTS

Method Syntax

```
1 _sql_constraints = [  
2     ('name_unique', 'unique(name)', 'Student name must be unique.'),  
3     ('age_positive', 'CHECK(age >= 0)', 'Age must be positive.'),  
4 ]
```

- ▶ Each tuple is: (name, sql_definition, error_message).
- ▶ Enforced by PostgreSQL.
- ▶ Excellent for uniqueness and simple CHECK rules.

PYTHON VS SQL CONSTRAINTS

► Python

- Flexible logic
- Better dynamic messages
- Can use related fields / ORM searches

► SQL

- Strongest guarantee
- Faster low-level checks
- Limited to SQL expressions

PRACTICAL EXAMPLE TOGETHER

```
1 class Student(models.Model):
2     _name = 'student.student'
3
4     email = fields.Char()
5     age = fields.Integer()
6
7     _sql_constraints = [
8         ('email-unique', 'unique(email)', 'Email must be unique.'),
9     ]
10
11     @api.constrains('age')
12     def _check_age(self):
13         for student in self:
14             if student.age and student.age > 100:
15                 raise ValidationError("Age seems invalid.")
```

Extension and Delegation Inheritance

TWO INHERITANCE STYLES

- ▶ Odoo model inheritance has two major styles:
 - 1 **Extension inheritance** with `_inherit`
 - 2 **Delegation inheritance** with `_inherits`
- ▶ They solve different problems.
- ▶ Understanding both is essential for clean module design.

EXTENSION INHERITANCE (`_INHERIT`)

- ▶ Extends an existing model in place.
- ▶ Adds fields/methods to the same model/table.

Method Syntax

```
1 class Student(models.Model):  
2     _inherit = 'student.student'  
3  
4     blood_group = fields.Char()
```

- ▶ No new model name is required when you only extend.

CLASSICAL EXTENSION PATTERNS

```
1 # 1) Extend existing model
2 class ResPartner(models.Model):
3     _inherit = 'res.partner'
4     is_teacher = fields.Boolean()
5
6 # 2) Prototype inheritance: new model based on another
7 class StudentAlumni(models.Model):
8     _name = 'student.alumni'
9     _inherit = 'student.student'
```

- ▶ `_inherit` alone extends the original model.
- ▶ `_name + _inherit` creates a new model copying the parent definition.

DELEGATION INHERITANCE (`_INHERITS`)

- ▶ Creates a new model that **delegates** fields to a linked parent record.
- ▶ Classic example: `res.users` delegates to `res.partner`.

Method Syntax

```
1 class StudentProfile(models.Model):  
2     _name = 'student.profile'  
3     _inherits = {'res.partner': 'partner_id'}  
4  
5     partner_id = fields.Many2one(  
6         'res.partner', required=True, ondelete='cascade')  
7     admission_no = fields.Char()
```

HOW DELEGATION WORKS

```
1 profile = self.env[ 'student.profile' ].create({  
2     'name': 'Ahmed Muhumed',      # goes to res.partner  
3     'email': 'ahmed@example.com',  # goes to res.partner  
4     'admission_no': 'A-001',      # local field  
5 })  
6 print(profile.name)    # delegated field  
7 print(profile.partner_id)
```

- ▶ Delegated fields are readable/writable through the child model.
- ▶ Data is stored in the parent table.

EXTENSION VS DELEGATION

- ▶ Use **extension** when you want to customize an existing model.
- ▶ Use **delegation** when you want a new model that reuses another model's data compositionally.

```
1 _inherit = 'student.student'           # extend
2 _inherits = { 'res.partner': 'partner_id' } # delegate
```

- ▶ Do not confuse `_inherit` with `_inherits`.

INHERITANCE BEST PRACTICES

- ▶ Prefer extension for small feature additions.
- ▶ Call `super()` when overriding methods.
- ▶ Keep delegated `Many2one` required with a clear `ondelete`.
- ▶ Avoid deep inheritance chains that are hard to debug.

XPath and Extend Views

WHAT IS VIEW INHERITANCE?

- ▶ View inheritance lets you modify an existing view without rewriting it.
- ▶ You create a new `ir.ui.view` with:
 - `inherit_id` pointing to the parent view
 - XPath / `<field name="...">` instructions describing changes
- ▶ This is the standard way to customize Odoo screens.

BASIC INHERITED VIEW STRUCTURE

```
1 <record id="view_student_form_inherit" model="ir.ui.view">
2   <field name="name">student.student.form.inherit</field>
3   <field name="model">student.student</field>
4   <field name="inherit_id" ref="school_manager.view_student_form"/>
5   <field name="arch" type="xml">
6     <!-- modification instructions here -->
7   </field>
8 </record>
```

SHORTCUT: INHERIT BY FIELD NAME

```
1 <field name="email" position="after">  
2   <field name="blood_group"/>  
3 </field>
```

- ▶ This finds the field named `email` and applies a position.
- ▶ Common positions:
 - `after`
 - `before`
 - `inside`
 - `replace`
 - `attributes`

Method Syntax

```
1 <xpath expr="//field[@name='age']" position="after">  
2   <field name="birth_date"/>  
3 </xpath>
```

- ▶ `expr` is an XPath expression into the parent architecture.
- ▶ More powerful when the target is not a simple field.

USEFUL XPATH EXAMPLES

```
1 <!-- Add a page inside a notebook -->
2 <xpath expr="//notebook" position="inside">
3   <page string="Medical">
4     <field name="blood_group"/>
5   </page>
6 </xpath>
7
8 <!-- Replace a field -->
9 <xpath expr="//field[@name='notes']" position="replace">
10   <field name="internal_notes"/>
11 </xpath>
12
13 <!-- Change attributes -->
14 <xpath expr="//field[@name='email']" position="attributes">
15   <attribute name="required">1</attribute>
16 </xpath>
```


POSITIONS EXPLAINED

- ▶ `before`: insert before the matched node
- ▶ `after`: insert after the matched node
- ▶ `inside`: insert as a child of the matched node
- ▶ `replace`: replace the matched node completely
- ▶ `attributes`: modify XML attributes of the matched node

VIEW EXTENSION TIPS

- ▶ Target stable nodes (named fields, named groups/pages).
- ▶ Prefer precise XPath expressions.
- ▶ Do not replace large view chunks unless necessary.
- ▶ Test after module upgrade because parent views may change.

```
1 <xpath expr="//page[@name='info']" position="inside">
2   <group>
3     <field name="guardian_phone"/>
4   </group>
5 </xpath>
```

Smart / Stats Buttons

WHAT ARE SMART BUTTONS?

- ▶ Smart buttons (stat buttons) appear at the top of a form view.
- ▶ They show a KPI / count and open related records when clicked.
- ▶ Excellent for navigation and dashboard-like form headers.

STAT BUTTON IN THE FORM VIEW

```
1 <div class="oe_button_box" name="button_box">
2   <button class="oe_stat_button"
3     type="object"
4     name="action_view_attendances"
5     icon="fa-calendar-check-o">
6   <field name="attendance_count" widget="statinfo" string="Attendances"/>
7   </button>
8 </div>
```

- ▶ Place them inside `oe_button_box`.
- ▶ Use `statinfo` widget for the count/value.

COMPUTED COUNT FIELD

```
1 attendance_count = fields.Integer(compute='_compute_attendance_count')
2
3 def _compute_attendance_count(self):
4     Attendance = self.env['student.attendance']
5     for student in self:
6         student.attendance_count = Attendance.search_count([
7             ('student_id', '=', student.id),
8         ])
```

- The button label usually comes from this computed field.

BUTTON ACTION METHOD

```
1 def action_view_attendances(self):  
2     self.ensure_one()  
3     return {  
4         'type': 'ir.actions.act_window',  
5         'name': 'Attendances',  
6         'res_model': 'student.attendance',  
7         'view_mode': 'list,form',  
8         'domain': [('student_id', '=', self.id)],  
9         'context': {'default_student_id': self.id},  
10    }
```

- ▶ The action opens related records filtered to the current student.
- ▶ Context can set defaults for creating new related rows.

ADDING A SMART BUTTON BY INHERITANCE

```
1 <xpath expr="//div[@name='button_box']" position="inside">
2   <button class="oe_stat.button" type="object"
3     name="action_view_attendances"
4     icon="fa-calendar-check-o">
5   <field name="attendance_count" widget="statinfo" string="Attendances"/>
6   </button>
7 </xpath>
```


SMART BUTTON BEST PRACTICES

- ▶ One button = one related document type / KPI.
- ▶ Keep count compute methods efficient.
- ▶ Use clear icons and short labels.
- ▶ Always return an action with a proper domain.
- ▶ Call `ensure_one()` in the button method.

Advanced Views

BEYOND FORM AND LIST

- ▶ Form and list/tree are only the beginning.
- ▶ Advanced views help users analyze and organize records visually.
- ▶ In this section:
 - Kanban
 - Pivot
 - Graph
 - Calendar
 - Activity
 - Gantt
 - Search view reminders for all of them

DECLARING MULTIPLE VIEW MODES

```
1 <record id="action_student" model="ir.actions.act_window">
2   <field name="name">Students</field>
3   <field name="res_model">student.student</field>
4   <field name="view_mode">kanban,list,form,pivot,graph,calendar,activity</field>
5 </record>
```

- ▶ The order in `view_mode` controls the switcher order.
- ▶ Each mode needs a corresponding view architecture.

KANBAN VIEW

- Kanban shows records as cards, often grouped in columns.

```
1 <kanban default_group_by="classroom_id" class="o_kanban_small_column">
2   <field name="name"/>
3   <field name="classroom_id"/>
4   <field name="age"/>
5   <templates>
6     <t t-name="card">
7       <div class="fw-bold"><field name="name"/></div>
8       <div>Age: <field name="age"/></div>
9     </t>
10  </templates>
11 </kanban>
```

- Great for pipelines, classrooms, stages, and boards.

- ▶ Use `default_group_by` for column grouping.
- ▶ Keep cards compact: title + a few key fields.
- ▶ Include fields used in the template as `<field/>` declarations.
- ▶ In Odoo 17, card templates often use `t-name="card"`.

PIVOT VIEW

- ▶ Pivot is for interactive spreadsheet-like analysis.

```
1 <pivot string="Student Analysis">  
2   <field name="classroom_id" type="row"/>  
3   <field name="gender" type="col"/>  
4   <field name="age" type="measure"/>  
5 </pivot>
```

- ▶ type="row" / col" define axes.
- ▶ type="measure" defines aggregated values.

- ▶ Graph shows bar/line/pie charts.

```
1 <graph string="Students by Classroom" type="bar">  
2   <field name="classroom_id"/>  
3   <field name="age" type="measure"/>  
4 </graph>
```

- ▶ Useful for dashboards and quick comparisons.
- ▶ Common types: bar, line, pie.

CALENDAR VIEW

- ▶ Calendar places records on dates.

```
1 <calendar string="Student Events"  
2   date_start="start_datetime"  
3   date_stop="end_datetime"  
4   color="classroom_id"  
5   mode="month">  
6   <field name="name"/>  
7   <field name="classroom_id"/>  
8 </calendar>
```

- ▶ Needs at least a start datetime/date field.

ACTIVITY VIEW

- ▶ Activity view focuses on scheduled activities and follow-ups.

```
1 <activity string="Student Activities">  
2   <field name="name"/>  
3   <field name="activity_state"/>  
4   <field name="activity_ids"/>  
5 </activity>
```

- ▶ Usually used with `mail.activity.mixin`.
- ▶ Excellent for CRM-like follow-up workflows.

GANTT VIEW

- ▶ Gantt shows time ranges on a timeline (Enterprise in many Odoo setups).

```
1 <gantt string="Classroom Schedule"  
2   date_start="start_datetime"  
3   date_stop="end_datetime"  
4   default_group_by="classroom_id"  
5   color="teacher_id">  
6   <field name="name"/>  
7 </gantt>
```

- ▶ Ideal for planning, allocation, and scheduling.

SEARCH VIEW SUPPORT FOR ADVANCED VIEWS

```
1 <search>
2   <field name="name"/>
3   <field name="classroom_id"/>
4   <filter string="Adults" name="adults" domain="['age','>=','18']"/>
5   <group>
6     <filter string="Classroom" name="group_classroom"
7       context="{ 'group_by': 'classroom_id' }"/>
8     <filter string="Gender" name="group_gender"
9       context="{ 'group_by': 'gender' }"/>
10  </group>
11 </search>
```

- ▶ Group-by filters are especially important for pivot/graph/kanban.

CHOOSING THE RIGHT ADVANCED VIEW

- ▶ **Kanban:** boards / stages / cards
- ▶ **Pivot:** analytical tables
- ▶ **Graph:** charts and trends
- ▶ **Calendar:** date-based events
- ▶ **Activity:** follow-up management
- ▶ **Gantt:** schedules and allocations

Navigation and Actions

HOW USERS REACH YOUR VIEWS

- ▶ Menus and window actions connect models to screens.
- ▶ A typical chain is:
 - 1 Menu item
 - 2 Window action (`ir.actions.act_window`)
 - 3 View modes + views
- ▶ Smart buttons and wizards also return actions for navigation.

WINDOW ACTION RECAP

```
1 <record id="action_student" model="ir.actions.act_window">
2   <field name="name">Students</field>
3   <field name="res_model">student.student</field>
4   <field name="view_mode">list,kanban,form,graph,pivot</field>
5   <field name="context">{}</field>
6   <field name="domain">[('active', '=', True)]</field>
7 </record>
```



```
1 <menuitem id="menu_school_root" name="School"/>
2 <menuitem id="menu_students"
3     name="Students"
4     parent="menu_school_root"
5     action="action_student"
6     sequence="10"/>
```

- ▶ `parent` builds the menu tree.
- ▶ `action` opens the window action.

NAVIGATION FROM PYTHON

```
1 def action_open_classroom(self):  
2     self.ensure_one()  
3     return {  
4         'type': 'ir.actions.act_window',  
5         'name': 'Classroom',  
6         'res_model': 'school.classroom',  
7         'view_mode': 'form',  
8         'res_id': self.classroom_id.id,  
9         'target': 'current',  
10    }
```

- ▶ target: current, new, fullscreen, etc.
- ▶ Buttons, smart buttons, and wizards all use this pattern.

NAVIGATION BEST PRACTICES

- ▶ Keep menu labels clear and short.
- ▶ Put the most common view mode first.
- ▶ Use domains/context in actions instead of hard-coding filters in every method.
- ▶ Reuse actions from smart buttons and wizards when possible.

Summary

CHAPTER FOUR SUMMARY

- ▶ **Transient models** power wizard forms and temporary processes.
- ▶ **Abstract models** share reusable mixin logic.
- ▶ **Python + SQL constraints** protect data quality.
- ▶ **`_inherit`** extends models; **`_inherits`** delegates to a parent record.
- ▶ **XPath / view inheritance** customizes screens cleanly.
- ▶ **Smart buttons** expose KPIs and related documents.
- ▶ **Advanced views** (Kanban, Pivot, Graph, Calendar, Activity, Gantt) unlock richer UX.

PRACTICAL BUILD ORDER

- 1 Create/extend the model and constraints
- 2 Add form/list views
- 3 Inherit views with XPath where needed
- 4 Add smart buttons and actions
- 5 Add advanced views and search group-bys
- 6 Add wizards for multi-step processes

CHECKLIST

- ▶ Wizard? → `TransientModel` + **form** + `target='new'`
- ▶ Shared logic? → `AbstractModel` **mixin**
- ▶ Validation? → `@api.constrains` **and/or** `_sql_constraints`
- ▶ Extend model/view? → `_inherit` / `XPath`
- ▶ Compose partner-like data? → `_inherits`
- ▶ KPI shortcut? → **smart button** + **action**
- ▶ Analysis/schedule UI? → `pivot/graph/calendar/gantt/kanban/activity`